

Выбор датасета

Использовался **YouTube-8M** — открытый датасет от Google для классификации видео.

 <https://research.google.com/youtube8m/>

Что в нём есть

- **~6 млн** YouTube-видео
- **3800 меток** — категории контента (типа Video Game, Music, Cooking, Sports car, Concert...). Каждое видео помечено сразу несколькими метками: например, видео по Minecraft помечено как Video Game, Minecraft и Sandbox game одновременно
- В работе из 3800 меток выбраны только релевантные и сгруппированы в **12 жанров**: Animals, Animation, Beauty, Dance, Film, Food, Gaming, Music, Performance, Sports, Tech, Vehicles

Формат данных

Каждое видео не лежит как .mp4, а представлено набором числовых признаков:

Поле	Размер	Описание
id	string	анонимизированный ID YouTube-видео
labels	int64 (мультилейбл)	список меток (индексы из 3800 категорий)
rgb	T × 1024 uint8	визуальные признаки кадра (Inception-v3+PCA)
audio	T × 128 uint8	аудио-признаки кадра (VGGish+PCA)

Длина T ≤ 300 (видео до 5 минут, 1 кадр = 1 секунда). Данные хранятся в TFRecord.

Использованный объём

Полный frame-level датасет — ~31 GB и тысячи shard-ов. В работе скачано и обработано **100 shard-файлов** (validate*.tfrecord) — этого хватило для получения ~22 тыс. видео до балансировки и ~10 тыс. после.

Парсинг данных

Из ~100 скачанных TFRecord-файлов собрали обучающую выборку.

1. Распарсили каждое видео как `tf.train.SequenceExample`, вытащили `rgb`, `audio`, `labels` и `id`
2. Каждой YT8M-метке сопоставили один из 12 жанров через словарь LABEL_MAP (всего отображено ~50 наиболее релевантных меток из 3800)
3. Если у видео несколько подходящих меток — берётся самый редкий жанр (приоритет: Beauty > Film > Dance > ... > Music > Gaming). Это нужно, чтобы редкие классы не "съедались" частыми
4. Каждое видео обрезано/паддировано до фиксированной длины **L = 60 кадров** (= 60 секунд). Длиннее видео обрезаются, короче — дополняются нулями. Реальная длина сохраняется отдельно для маски в RNN
5. Данные хранятся как `uint8` (нативная квантизация YT8M) — экономит память в 4 раза. Деквантизация в `float` делается на лету при обучении

Что получилось до балансировки

- Всего видео с подходящими метками: **22 218**
- Пропущено (не было нужных меток / пустое видео): **7 128**
- Сильный дисбаланс классов: Gaming и Music ~22%, Beauty ~1.4%
- Imbalance ratio: **15.5×** (Gaming vs Beauty)

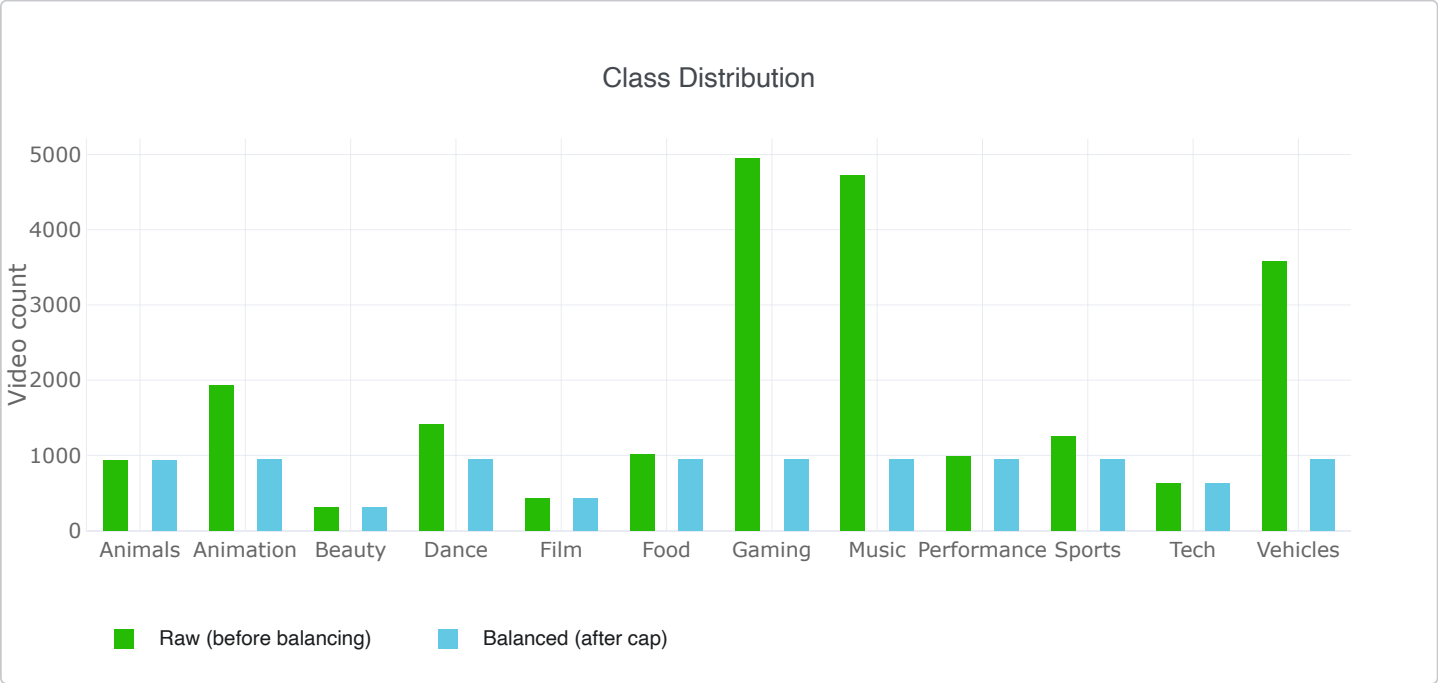
Балансировка

Чтобы избежать перекоса в сторону Gaming/Music применили **cap per class**:

$$\text{MAX_PER_CLASS} = \min(5000, \min_class_count \times 3) = \min(5000, 319 \times 3) = 957$$

Из каждого класса оставили максимум 957 примеров. Итог:

- **9 987 видео** после балансировки
- Imbalance ratio: **3.0×**
- 12 классов



На графике видно, что Gaming и Music доминируют (~5000 и ~4700 видео), тогда как Beauty — всего 319. После балансировки все классы выровнены до ~957, кроме тех, что изначально были меньше порога (Beauty, Film, Tech, Animals) — они остались как есть.

Разбивка на выборки

Данные разбиты на три выборки стратифицированно — соотношение классов сохраняется в каждой:

Выборка	Размер	Назначение
Train	6 990	Обучение модели
Val	1 498	Подбор гиперпараметров, early stopping
Test	1 499	Финальная оценка (не трогается до конца)

Нормализация

Для каждого признака отдельно вычислены среднее и стандартное отклонение **только по train**. Статистика считается по реальным кадрам (падинг не учитывается), затем сохраняется и применяется одинаково ко всем трём выборкам.

Паддинговые кадры (где T < 60) остаются нулями — RNN игнорирует их через маску длин.

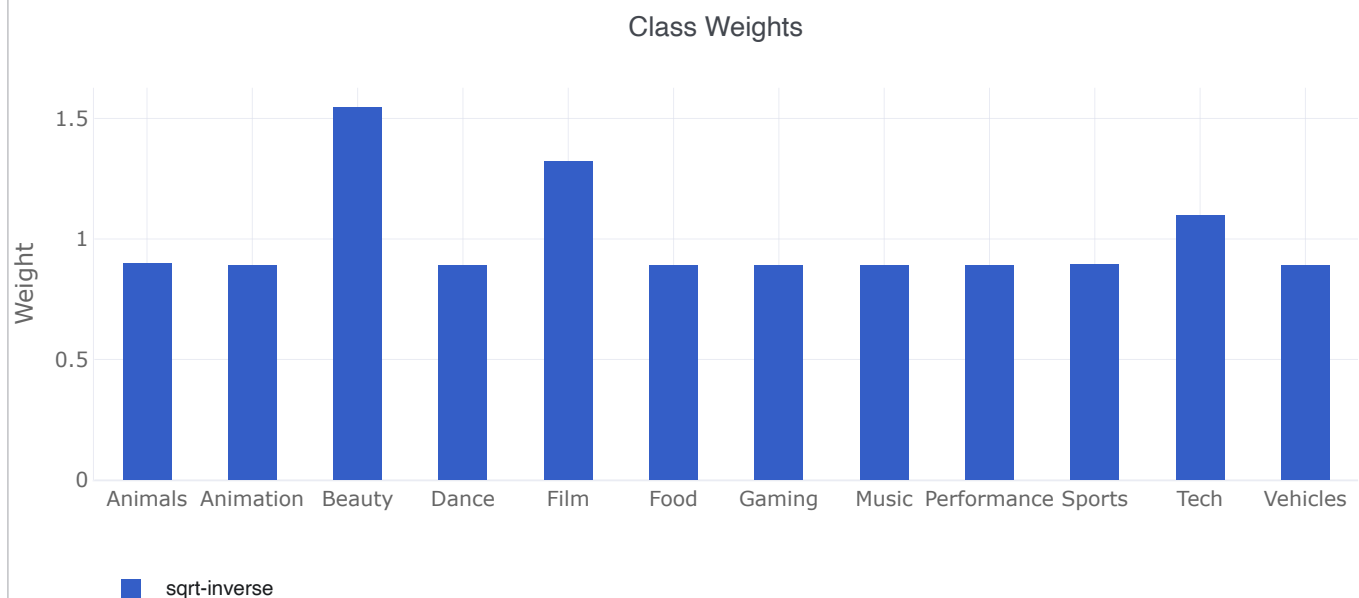
Веса классов

Чтобы редкие классы (Beauty, Film) не тонули в loss на фоне частых, применены **sqrt-inverse** веса:

$$\text{weight}[c] = 1 / \text{sqrt}(\text{count}[c])$$

После нормализации веса центрированы так, что их сумма равна числу классов. Итог: Beauty получает вес ~1.55, Gaming и Music — ~0.89.

Summary		
n_test	n_train	n_val
1499	6990	1498



Модели и обучение

Архитектуры

Сравнивались три подхода к классификации видео по последовательности кадров.

FlattenMLP

Самый простой вариант — берём первые **30 кадров**, конкатенируем rgb+audio для каждого и сплющиваем в один длинный вектор → MLP с двумя скрытыми слоями (1024 → 512 → 12). Никакой обработки временной структуры нет.

- Вход: (B, 30, 1152) → flatten → (B, 34 560)
- Параметров: **35.9M**

FrameRNN

Каждый кадр проецируется в 256-мерное пространство, затем прогоняется через **двунаправленный GRU**. Паддинг маскируется через `pack_padded_sequence` — модель не тратит вычисления на нулевые кадры.

- Вход: (B, L, 1152) → Linear(256) → biGRU(256) → (B, 512) → Linear(12)
- Параметров: **1.09M**

GatedRNN

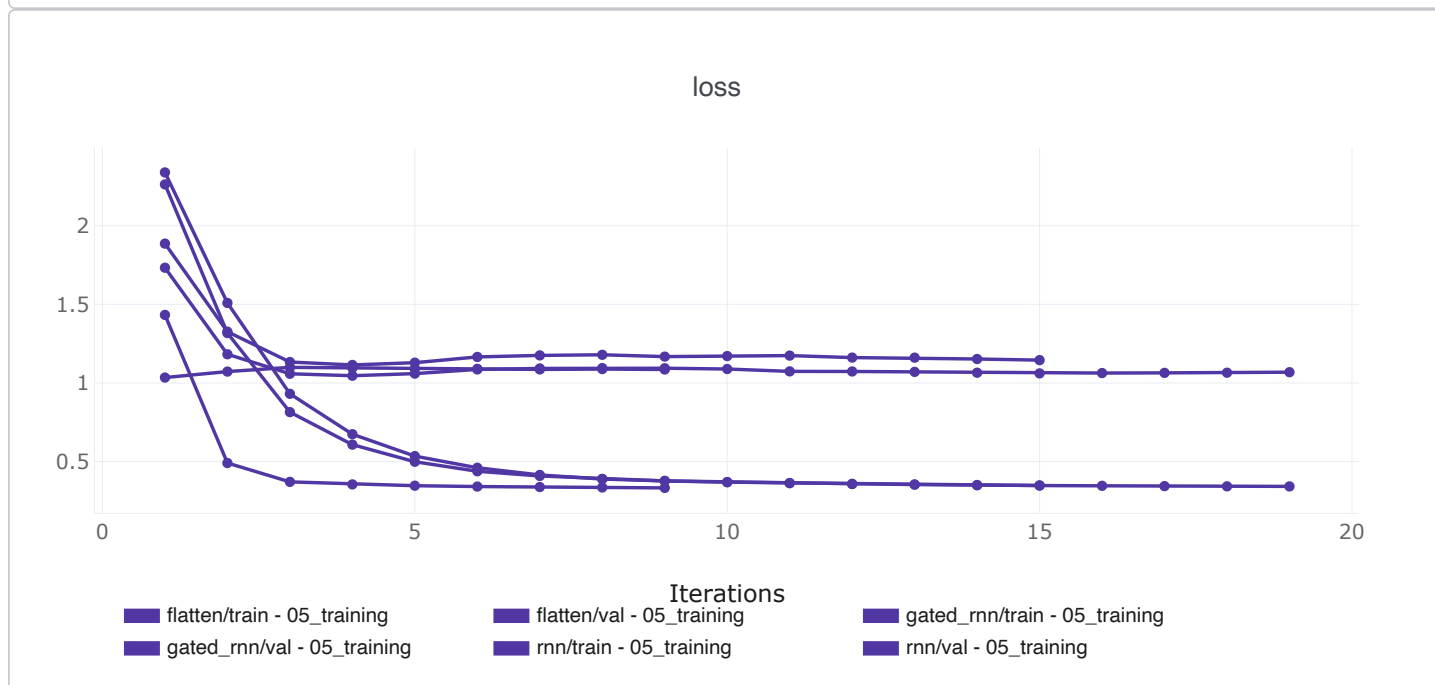
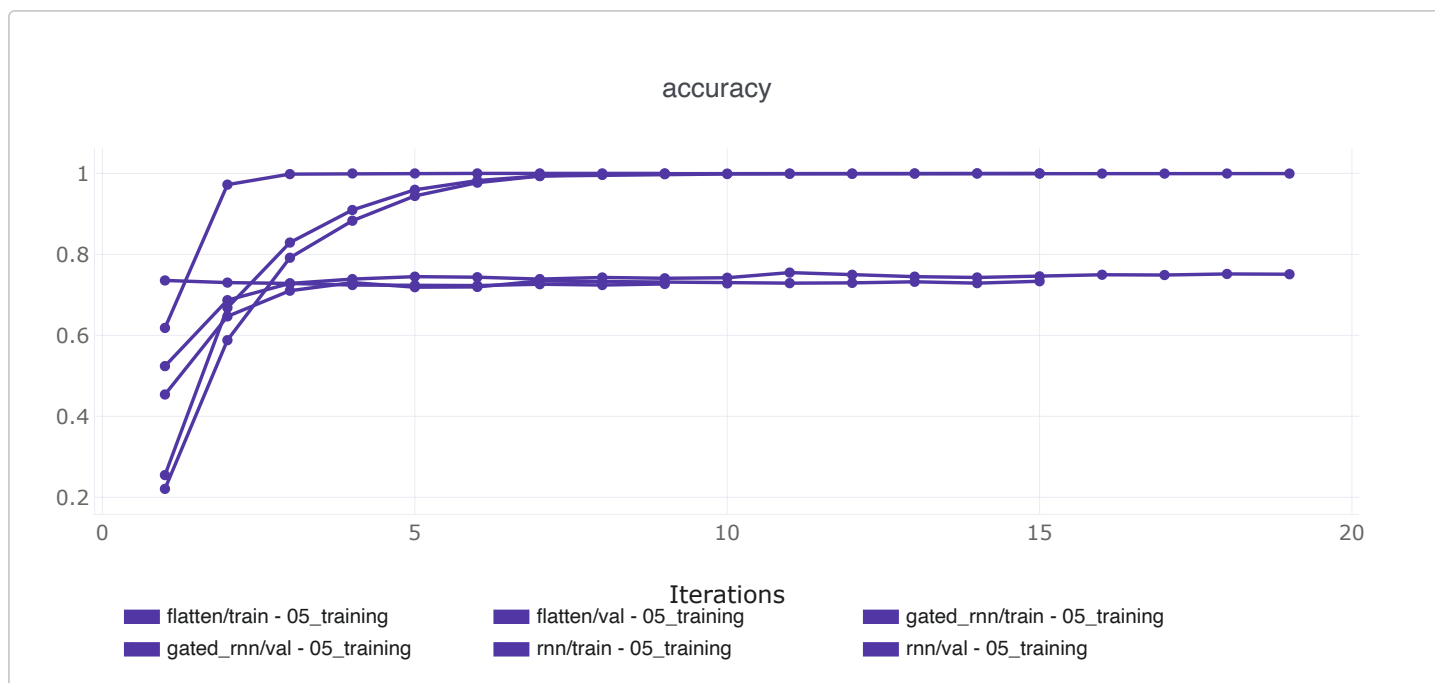
Расширение FrameRNN: отдельная сеть **AudioGateNet** (25K параметров) решает для каждого видео, полезно ли аудио (0 или 1). Аудио умножается на gate, а скаляр gate добавляется к представлению перед классификатором.

- Параметров: **1.09M** + 25K (gate)

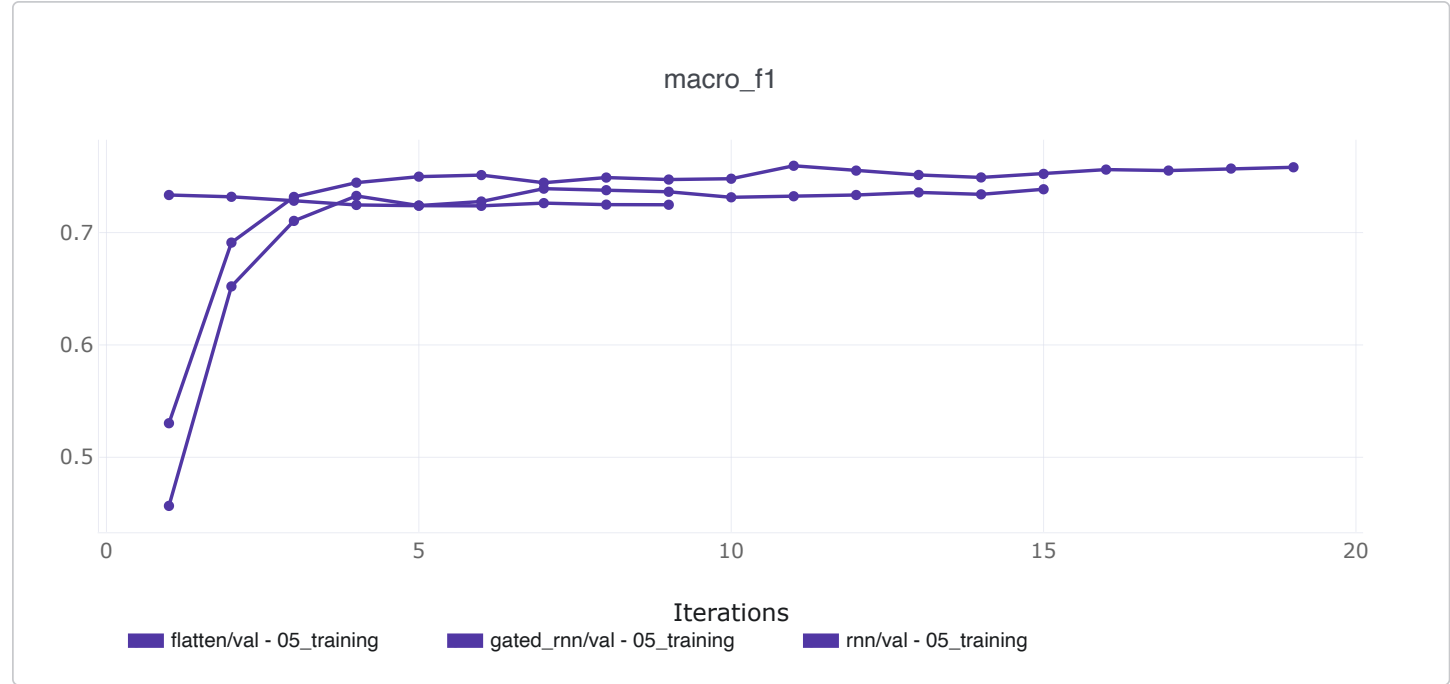
Обучение

Все три модели обучались на одинаковых данных с одинаковыми настройками:

Параметр	Значение
Оптимизатор	AdamW
Learning rate	3e-4 (cosine decay)
Batch size	256
Epochs	25 (max)
Early stopping	по val macro-F1, patience=8
Loss	CrossEntropy + label smoothing 0.05 + class weights
Аугментация	Gaussian noise на кадры + drop audio 10%



На графиках видно ключевую разницу: **FlattenMLP** уже на 2-й эпохе достигает accuracy=1.0 на train — модель просто запоминает обучающую выборку. Val accuracy при этом не растёт. **FrameRNN** и **GatedRNN** учатся плавнее, val loss продолжает снижаться несколько эпох.

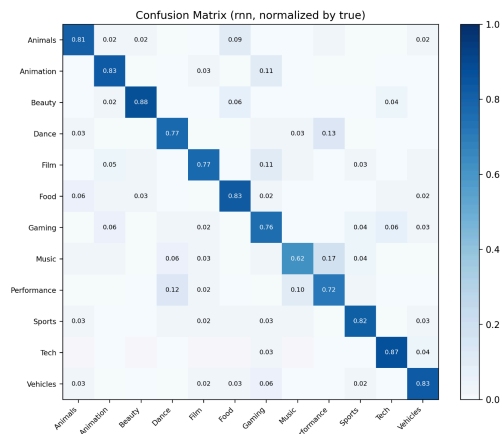


По macro-F1 на val лучший результат у **FrameRNN** — метрика стабильно растёт до эпохи 11, после чего срабатывает early stopping.

Результаты на тестовой выборке

Модель	Test Accuracy	Test Macro-F1
FlattenMLP	0.769	0.769
GatedRNN	0.757	0.753
FrameRNN	0.785	0.784

Лучшей оказалась **FrameRNN** — несмотря на в 33 раза меньше параметров, чем у FlattenMLP. Это подтверждает, что временная структура видео несёт важную информацию, которую flatten теряет.



Inference

Пайплайн извлечения признаков

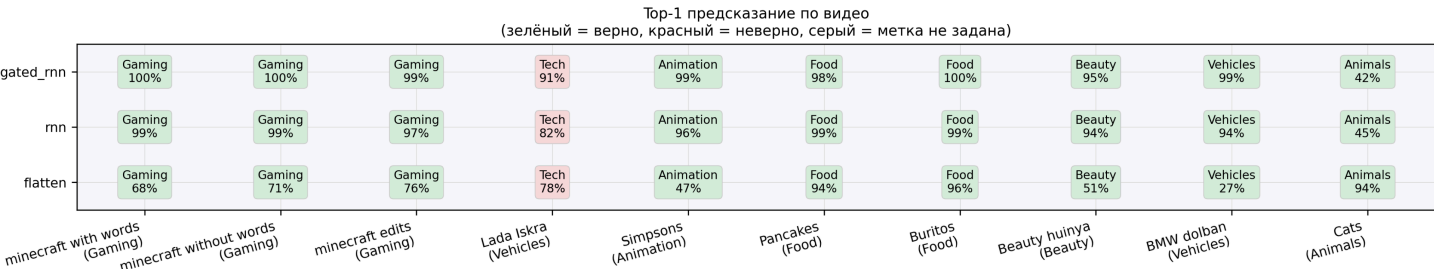
Для классификации произвольного видео реализован следующий пайплайн:

1. **Загрузка видео** — утилита `yt-dlp` загружает видео с YouTube длительностью до 60 секунд в формате mp4 (кодек H.264)

- 2. **Сэмплирование кадров** — из видеопотока извлекается **1 кадр/сек** посредством OpenCV, что соответствует гранулярности признаков YT8M
- 3. **Извлечение визуальных признаков** — каждый кадр обрабатывается предобученной сетью Inception-v3, активации предпоследнего слоя (2048D) проецируются в 1024-мерное пространство через PCA-матрицу YouTube-8M
- 4. **Аудио-признаки** — оригинальные аудио-эмбединги YT8M получены моделью VGGish, которая не имеет публично доступных весов. В данной работе аудио-канал обнуляется; устойчивость модели к отсутствию аудио обеспечивается аугментацией `drop_audio_p=0.1` на этапе обучения
- 5. **Нормализация** — признаки нормализуются статистиками, вычисленными на обучающей выборке (`norm_stats.npz`)
- 6. **Классификация** — последовательность кадров подаётся в обученную модель, на выходе — распределение вероятностей по 12 жанрам

Сравнение моделей на реальных видео

Для оценки качества на практике отобрано **11 коротких роликов** с YouTube, охватывающих жанры: Gaming, Vehicles, Animation, Food, Animals, Beauty. Каждое видео классифицировалось всеми тремя моделями.



FlattenMLP, FrameRNN и GatedRNN демонстрируют высокую уверенность на Gaming-контенте (98–99%), что объясняется его визуальной однородностью и преобладанием в обучающей выборке до балансировки. На менее представленных жанрах FrameRNN показывает наибольшую стабильность предсказаний.